

Designing a RAG-Based Q&A System for News Retrieval

Andri Gerber, Kened Kqiraj, Amir Shatrolli

Department of Business, Lucerne University of Applied Sciences and Arts

2025-04-23

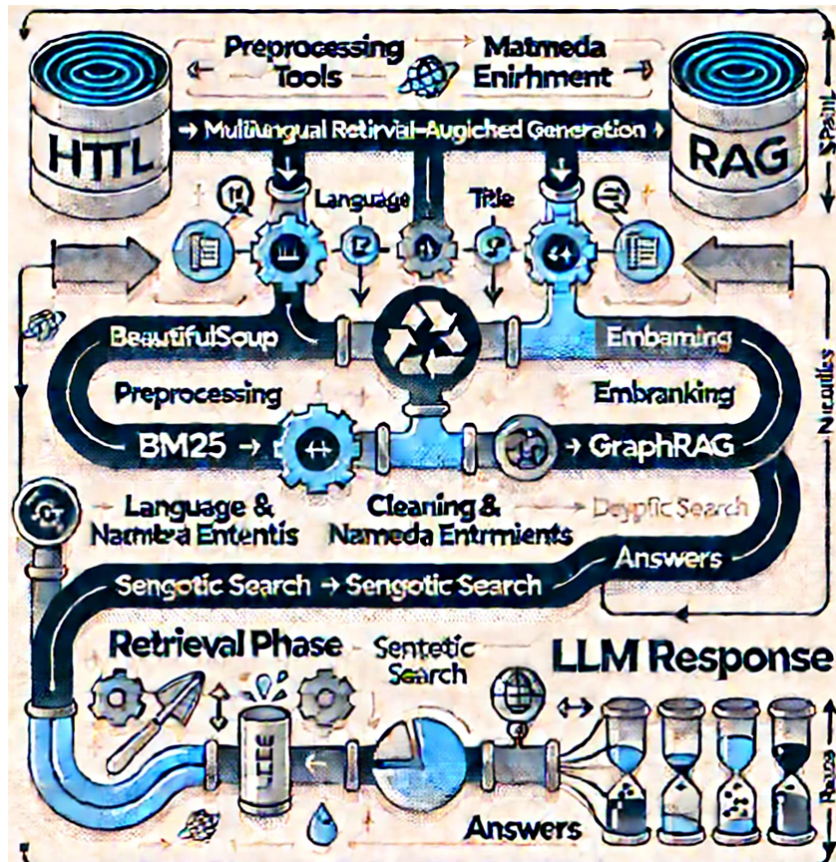
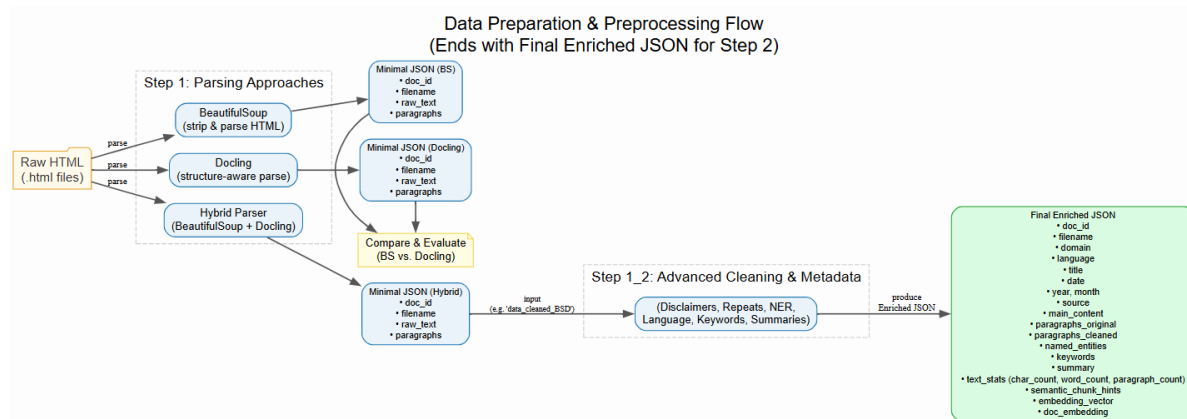


Table of contents

1	General Overview	3
2	Step 1_1 Data Preparation – Structuring the Dataset	3
2.1	Background & Objectives	3
2.2	Environment Setup	3
2.3	Pipeline Invocation	4
2.4	Detailed Code Walk-through	5
2.4.1	BeautifulSoup Script Highlights	5
2.4.2	Docling Script Highlights	5
3	BeautifulSoup vs. Docling	5
3.1	Configuration & Imports	5
3.1.1	Helper Function: Loading the JSON Files	6
3.1.2	Checking for Duplicate in <code>doc_id</code> 's for each folder	7
3.1.3	Load Both Datasets	7
3.1.4	Compute Comparison Metrics	8
3.1.5	Aggregate Patterns Across the Corpus	10
3.1.6	Side-by-Side Text Snippet	13
3.1.7	Pros & Cons Table	15
3.1.8	Conclusion	15
4	Step 1_2 Preprocessing	16
4.1	Introduction	16
4.2	Hybrid Parsing with <code>step_1_hybrid.py</code>	16
4.2.1	Rationale	16
4.2.2	Workflow	17
4.2.3	Key Advantages	17
4.3	Advanced Cleaning & Metadata with <code>step_1_2_advanced_cleaning_and_metadata.py</code>	17
4.3.1	Purpose	17
4.3.2	Detailed Process	18
4.3.3	Advantages of a Two-Step Approach	19
4.4	Putting It All Together	20
4.4.1	3.1 End-to-End Flow	20
4.4.2	Script Usage	20
4.4.3	Outcome	20
5	Preliminary Next Steps Toward RAG Implementation (not done yet)	20
5.1	Validate & Finalize the Structured Dataset	21
5.2	Chunking & Preprocessing for Retrieval	21
5.3	Index Construction & Embeddings	21
5.4	Implement Baseline & Advanced Retrieval Agents	21
5.5	Hybrid Retrieval	22
5.6	Evaluation & Benchmarking	22
5.7	Final Report & Analysis	22

1 General Overview



2 Step 1_1 Data Preparation – Structuring the Dataset

2.1 Background & Objectives

In retrieval-augmented generation (RAG), the quality of retrieved documents directly influences answer accuracy. Hence:

1. Clean extraction of text from HTML is critical
2. Preserving structure (headings, paragraphs, tables) can improve downstream chunking and retrieval
3. We compare two parsing strategies:
 - **BeautifulSoup**: tag-stripping, minimal dependencies
 - **Docling**: semantic parsing, structure-aware, boilerplate removal

Goals

- Extract & normalize news articles in German & English
- Annotate each with `doc_id`, `filename`, `raw_text`, and `paragraphs`
- Evaluate output quality via character/word/paragraph counts and a token-based Jaccard metric
- Determine which parser is best for a production RAG pipeline

2.2 Environment Setup

We isolate dependencies to avoid conflicts and ensure reproducibility. Create a virtual environment python 3.11 and install the required libraries.

```
# 1) Activate your virtual environment
.venv\Scripts\activate
```

```
# 2) Install libraries
pip install bs4
pip install docling
pip install dateparser
pip install yake
pip install lingua-language-detector
pip install spacy
pip install nltk
```

```
# 3) Download models for spaCy and NLTK for Step 1_2
python -m spacy download en_core_web_sm
python -m spacy download de_core_news_sm
python -m nltk.downloader punkt
```

Why two parsers? - [BeautifulSoup](#) is ubiquitous, very fast, and ideal for quick “all-visible text” extraction. - [Docling](#) uses layout analysis to retain headings, drop nav bars, and collapse boilerplate—valuable for clean segmentation.

2.3 Pipeline Invocation

Assuming your raw HTML lives under `data/`, run:

1. **BeautifulSoup** usage: `python step_1_BeautifulSoup.py [input_folder] [output_folder]`

```
python Code/step_1_BeautifulSoup.py data data_cleaned/data_cleaned_BS
```

2. **Docling** usage: `python step_1_Docling.py [input_folder] [output_folder]`

```
python Code/step_1_Docling.py data data_cleaned/data_cleaned_D
```

Once these scripts finish, you should have two sets of JSON files:

- `data_cleaned_BS/`: output from the BeautifulSoup approach
- `data_cleaned_D/`: output from the Docling approach

Each script:

- Recursively finds `.html`
- Assigns a deterministic `doc_id` (SHA-1 of the relative path)
- Emits a JSON per document containing:

```
json { "doc_id": "...", "filename": "...", "raw_text": "...normalized text...", "paragraphs": [ "para 1", "para 2", ... ] }
```
- Logs progress and errors

2.4 Detailed Code Walk-through

2.4.1 BeautifulSoup Script Highlights

- `step_1_BeautifulSoup.py`:
 - **Tag removal**: We proactively drop elements known to contain non-article content.
 - **Normalization**: unicode NFC + strip blank lines.
 - **Paragraph split**: Split on two or more newlines, fallback to single-line paragraphs if only one chunk remains.

2.4.2 Docling Script Highlights

- `step_1_Docling.py`:
 - **DocumentConverter** analyzes layout, extracts headings, captions, and body text.
 - `export_to_text()` yields a structured plain-text representation (including `##` headings).

Trade-off: Docling startup is heavier (Java + layout libs), but output is cleaner.

3 BeautifulSoup vs. Docling

In this section, we perform a comprehensive comparison of two HTML-parsing pipelines—**BeautifulSoup** and **Docling**—to assess their output quality for downstream Retrieval-Augmented Generation (RAG). We load the cleaned JSON outputs, compute a suite of metrics on **4,388** shared documents, visualize corpus-wide trends, examine a representative text snippet, and conclude with detailed pros & cons.

3.1 Configuration & Imports

```
# Install libraries for comparison
pip install matplotlib
pip install seaborn
pip IPython
```

```
import argparse
import json
import logging
import re
from pathlib import Path
from typing import Dict, Set, Tuple

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from tqdm.auto import tqdm
from collections import defaultdict

# Enable inline plotting
%matplotlib inline

#-- CLI/config scaffolding --
```

```
parser = argparse.ArgumentParser(
    description="Compare BeautifulSoup vs Docling parsing outputs."
)
parser.add_argument(
    "--bs-dir",
    type=Path,
    default=Path("../data_cleaned/data_cleaned_BS"),
    help="Directory for BS JSON outputs",
)
parser.add_argument(
    "--dc-dir",
    type=Path,
    default=Path("../data_cleaned/data_cleaned_D"),
    help="Directory for Docling JSON outputs",
)
args, _ = parser.parse_known_args()

#-- Logging --
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s %(levelname)s %(message)s",
    datefmt="%H:%M:%S"
)
logger = logging.getLogger("compare_parsers")

#-- Matplotlib Defaults --
plt.rcParams.update({
    "figure.figsize": (10, 5),
    "font.size": 12,
    "axes.titlesize": 14,
    "axes.labelsize": 12,
})
COLOR_BS = "#1f77b4" # blue
COLOR_DOCL = "#d62728" # red
COLOR_JACC = "#2ca02c" # green
```

Note: Adjust the default paths (--bs-dir, --dc-dir) as needed to point to your actual data folders.

3.1.1 Helper Function: Loading the JSON Files

We'll load all JSON files in each directory into a Python dictionary keyed by `doc_id`.

```
## 2. Helper Functions for Loading JSON
def list_duplicates(json_dir: Path):
    """
    Returns a dict mapping doc_id -> list of Paths,
    including only those with more than one file.
    """
    mapping = defaultdict(list)
    for path in json_dir.rglob("*.json"):
        if not path.is_file():
            continue
        try:
            data = json.loads(path.read_text(encoding="utf-8"))
            doc_id = data.get("doc_id", path.stem)
        except Exception:
            # skip broken JSON
            continue
        mapping[doc_id].append(path)
    # filter to duplicates
    return {doc_id: paths for doc_id, paths in mapping.items() if len(paths) > 1}

def load_json_outputs(json_dir: Path) -> Dict[str, dict]:
    """
    Recursively find all .json paths under `json_dir`, then
    only attempt to read those that still exist. Returns
    a mapping doc_id -> loaded JSON dict.
    """
```

```
# 1) Gather all .json paths:
paths = list(json_dir.rglob("*.json"))
logger.info(f"Discovered {len(paths)} JSON paths under {json_dir!r}")

results: Dict[str, dict] = {}
for path in tqdm(paths, desc=f>Loading JSON from {json_dir.name}):
    # 2) Skip if missing
    if not path.is_file():
        logger.warning(f" • Skipping missing file: {path}")
        continue

    # 3) Attempt to parse
    try:
        raw = path.read_text(encoding="utf-8")
        data = json.loads(raw)
        doc_id = data.get("doc_id", path.stem)
        results[doc_id] = data
    except json.JSONDecodeError as e:
        logger.warning(f" • Invalid JSON in {path.name}: {e}")
    except Exception as e:
        logger.warning(f" • Error reading {path.name}: {e}")

logger.info(f"Loaded {len(results)} / {len(paths)} documents from {json_dir.name}")
return results
```

3.1.2 Checking for Duplicate in doc_id's for each folder

```
# Find and print duplicates in each folder
bs_dupes = list_duplicates(args.bs_dir)
dc_dupes = list_duplicates(args.dc_dir)

print(f"→ BeautifulSoup folder: {len(bs_dupes)} duplicate doc_ids\n")
for doc_id, paths in list(bs_dupes.items())[:5]:
    print(f" • {doc_id!r}:")
    for p in paths:
        print(f"   - {p}")
print(" ...")

print(f"→ Docling folder: {len(dc_dupes)} duplicate doc_ids\n")
for doc_id, paths in list(dc_dupes.items())[:5]:
    print(f" • {doc_id!r}:")
    for p in paths:
        print(f"   - {p}")
print(" ...")
```

→ BeautifulSoup folder: 0 duplicate doc_ids

...

→ Docling folder: 0 duplicate doc_ids

...

3.1.3 Load Both Datasets

```
## Robust Loader with Existence Check
bs_data = load_json_outputs(args.bs_dir)
dc_data = load_json_outputs(args.dc_dir)
```

```
common_ids = sorted(set(bs_data).intersection(dc_data))
logger.info(f"Found {len(common_ids)} documents in common between BS & Docling.")
```

```
INFO: Discovered 4390 JSON paths under WindowsPath('..data_cleaned/data_cleaned_BS')
Loading JSON from data_cleaned_BS: 48%|          | 2090/4390 [00:03<00:04, 529.94it/s]WARNING:  • Skipp
Loading JSON from data_cleaned_BS: 95%|          | 4163/4390 [00:07<00:00, 604.17it/s]WARNING:  • Skipp
Loading JSON from data_cleaned_BS: 100%|         | 4390/4390 [00:07<00:00, 562.87it/s]
INFO: Loaded 4388 / 4390 documents from data_cleaned_BS
INFO: Discovered 4390 JSON paths under WindowsPath('..data_cleaned/data_cleaned_D')
Loading JSON from data_cleaned_D: 48%|          | 2109/4390 [00:03<00:02, 774.60it/s]WARNING:  • Skipp
Loading JSON from data_cleaned_D: 94%|          | 4139/4390 [00:05<00:00, 773.74it/s]WARNING:  • Skipp
Loading JSON from data_cleaned_D: 100%|         | 4390/4390 [00:06<00:00, 713.40it/s]
INFO: Loaded 4388 / 4390 documents from data_cleaned_D
INFO: Found 4388 documents in common between BS & Docling.
```

Result: A total of 4,388 documents were successfully parsed by both pipelines, and no duplicate `doc_ids` were found in either output directory. We generated these `doc_ids` (via a SHA-1 hash of each file’s relative path) because the original HTML filenames weren’t globally unique across the English and German folders. We will later apply an automated deduplication step in our final hybrid parser to filter out duplicate or boilerplate paragraphs, ensuring each JSON contains only unique, non-redundant content.

3.1.4 Compute Comparison Metrics

For each document, we calculate four metrics to quantify and compare the HTML-parsing outputs:

- **Character count:** the total number of characters in the extracted text.
- **Word count:** the total number of whitespace-delimited tokens (words).
- **Paragraph count:** the number of segments produced by our paragraph-splitting logic.
- **Token-level Jaccard similarity:** a set-based measure of overlap between the unique tokens from BeautifulSoup and Docling.

Let

$$T_{BS} = \{\text{unique tokens from BeautifulSoup output}\}, \quad T_{Doc} = \{\text{unique tokens from Docling output}\}.$$

The **Jaccard similarity** (J) is then defined as:

$$J(T_{BS}, T_{Doc}) = \frac{|T_{BS} \cap T_{Doc}|}{|T_{BS} \cup T_{Doc}|},$$

where the numerator counts tokens common to both sets and the denominator counts all tokens present in either set. Values range from 0 (no overlap) to 1 (identical token sets) (1, 2).


```
## 5. Comparing Character, Word, Paragraph Counts + Jaccard Similarity
TOKEN_RE = re.compile(r"\w+", re.UNICODE)

def tokenize(text: str) -> Set[str]:
    """Lower-case alphanumeric tokens for Jaccard similarity."""
    return set(TOKEN_RE.findall(text.lower()))

def compute_metrics(bs_json: dict, docling_json: dict) -> Tuple[int, int, int, int, int, int, float]:
    """
    Compute a set of comparison metrics:
    - BS chars, Docling chars
    - BS words, Docling words
    - BS paragraph count, Docling paragraph count
    - Jaccard overlap of unique tokens
    """
    # Extract raw_text or fallback to paragraphs
    bs_text = bs_json.get("raw_text") or "\n".join(bs_json.get("paragraphs", []))
    dc_text = docling_json.get("raw_text") or "\n".join(docling_json.get("paragraphs", []))

    bs_chars = len(bs_text)
    dc_chars = len(dc_text)
    bs_words = len(bs_text.split())
    dc_words = len(dc_text.split())
    bs_pars = len(bs_json.get("paragraphs", []))
    dc_pars = len(docling_json.get("paragraphs", []))

    # Token-based Jaccard
    bs_tokens = tokenize(bs_text)
    dc_tokens = tokenize(dc_text)
    if not bs_tokens and not dc_tokens:
        jaccard = 1.0
    else:
        inter = bs_tokens & dc_tokens
        union = bs_tokens | dc_tokens
        jaccard = len(inter) / len(union) if union else 1.0

    return (bs_chars, dc_chars,
            bs_words, dc_words,
            bs_pars, dc_pars,
            jaccard)

# Gather metrics for each document
records = []
for doc_id in tqdm(common_ids, desc="Computing metrics"):
    rec_bs = bs_data[doc_id]
    rec_dc = dc_data[doc_id]
    (bs_chars, dc_chars,
     bs_words, dc_words,
     bs_pars, dc_pars,
     jaccard) = compute_metrics(rec_bs, rec_dc)

    records.append({
        "doc_id": doc_id,
        "bs_chars": bs_chars,
        "bs_words": bs_words,
        "bs_pars": bs_pars,
        "jaccard": jaccard,
        "docling_chars": dc_chars,
        "docling_words": dc_words,
        "docling_pars": dc_pars,
    })
}
```

Computing metrics: 100% | 4388/4388 [00:04<00:00, 962.03it/s]

On a per-document basis, we compute the following metrics:

```
df = pd.DataFrame(records).set_index("doc_id")
df.head(2)
```

doc_id	bs_chars	docling_chars	bs_words	docling_words	bs_pars	docling_pars
001c81229e64abbf81df56919a457b8cd112ee61	6562	5949	895	761	152	38
0026b764e2e3d7f092b448f7965c694901b1f1e7	6320	6359	850	859	98	59

3.1.4.1 Statistical Summary

```
logger.info("Summary statistics:")
df.describe().T
```

INFO: Summary statistics:

	count	mean	std	min	25%	50%	75%	max
bs_chars	4388.0	4144.937785	2673.224645	0.000000	1823.750000	4214.000000	5702.250000	21551.0
docling_chars	4388.0	3029.472653	2536.241213	0.000000	547.000000	2940.000000	4661.250000	22113.0
bs_words	4388.0	604.622607	393.138289	0.000000	263.000000	602.000000	836.000000	3420.0
docling_words	4388.0	439.097083	367.819041	0.000000	80.000000	422.000000	675.000000	3069.0
bs_pars	4388.0	24.738149	17.724278	0.000000	14.000000	21.000000	30.000000	195.0
docling_pars	4388.0	12.662944	9.279129	0.000000	4.000000	11.000000	18.000000	139.0
jaccard	4388.0	0.683043	0.289528	0.019874	0.445397	0.810028	0.891996	1.0

Interpretation:

The table shows that — on average — **BeautifulSoup** extracts **4,144.94** characters per document, whereas **Docling** extracts **3,029.47** characters, an absolute difference of 1,115.47 characters (36.8% more text in BeautifulSoup). Word counts mirror this pattern: BeautifulSoup yields **604.62** words versus **439.10** for Docling, a gain of **165.52** words (37.7% more).

However, BeautifulSoup’s naive paragraph-splitting produces **24.74** paragraphs on average—nearly **95.5%** more than Docling’s **12.66** paragraphs—indicating a tendency to over-segment at minor line breaks. In contrast, Docling’s output is more restrained, with roughly **27% fewer** characters and words but semantically **coherent** paragraph boundaries aligned to headings and logical breaks.

Finally, the mean **Jaccard similarity** of **0.683** implies that about **31.7%** of unique tokens differ between the two outputs, reflecting BeautifulSoup’s inclusion of boilerplate and incidental text versus Docling’s focused extraction of core content and structural markers.

3.1.5 Aggregate Patterns Across the Corpus

To verify that our observed differences are not artifacts of a particular subset, we partitioned the 4,388 documents into 44 equal “bins” of ~100 documents each and plotted three key metrics for each bin (Figures: 1–3).

- **Avg Word Count (Fig.1):** BeautifulSoup consistently yields **~530–690 words** per bin vs. **~340–510 words** for Docling, a stable gap of ~150–200 words—reflecting BeautifulSoup’s inclusion of boilerplate and incidental text.

```
# We'll create bins of size 100 docs each.
bin_size = 100

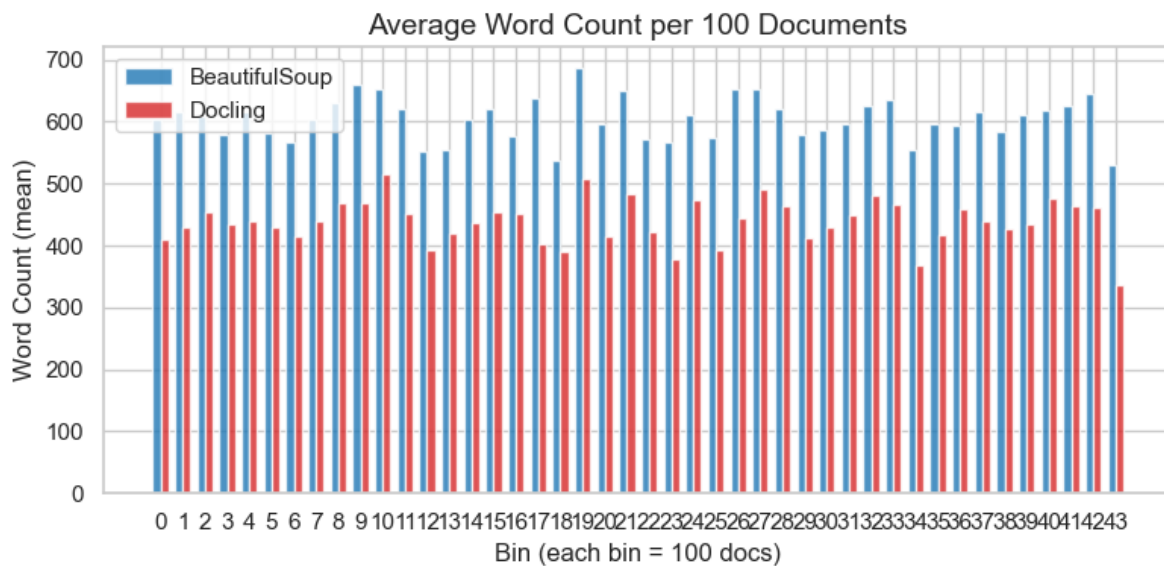
# 2) Assign a bin index to each row
df["bin"] = np.arange(len(df)) // bin_size

# 3) Compute mean values for each bin
bin_groups = df.groupby("bin").mean()

# 4) Create a bar plot
bins_index = np.arange(len(bin_groups)) # e.g. 0,1,2,... up to the number of bins
width = 0.35

plt.figure(figsize=(8,4))
plt.bar(
    bins_index - width/2,
    bin_groups["bs_words"],
    width,
    color="#1f77b4",
    alpha=0.8,
    label="BeautifulSoup",
)
plt.bar(
    bins_index + width/2,
    bin_groups["docling_words"],
    width,
    color="#d62728",
    alpha=0.8,
    label="Docling",
)

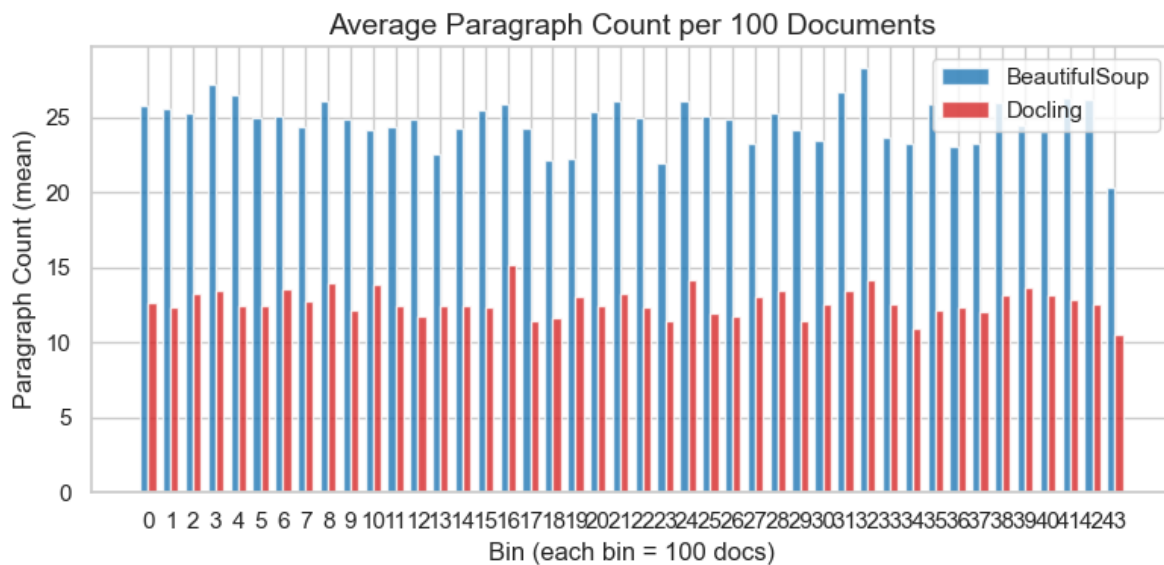
plt.title("Average Word Count per 100 Documents")
plt.xlabel("Bin (each bin = 100 docs)")
plt.ylabel("Word Count (mean)")
plt.xticks(bins_index) # Ensures bins appear as 0,1,2,...
plt.legend()
plt.tight_layout()
plt.show()
```



- **Avg Paragraph Count (Fig. 2):** BeautifulSoup produces ~20–28 paragraphs per bin, almost double Docling's ~10–15, due to naive splitting on every blank line; Docling's paragraphs align more closely with true semantic breaks.

```
# 5) Create a bar plot
plt.figure(figsize=(8,4))
plt.bar(
    bins_index - width/2,
    bin_groups["bs_pars"],
    width,
    color="#1f77b4",
    alpha=0.8,
    label="BeautifulSoup"
)
plt.bar(
    bins_index + width/2,
    bin_groups["docling_pars"],
    width,
    color="#d62728",
    alpha=0.8,
    label="Docling"
)

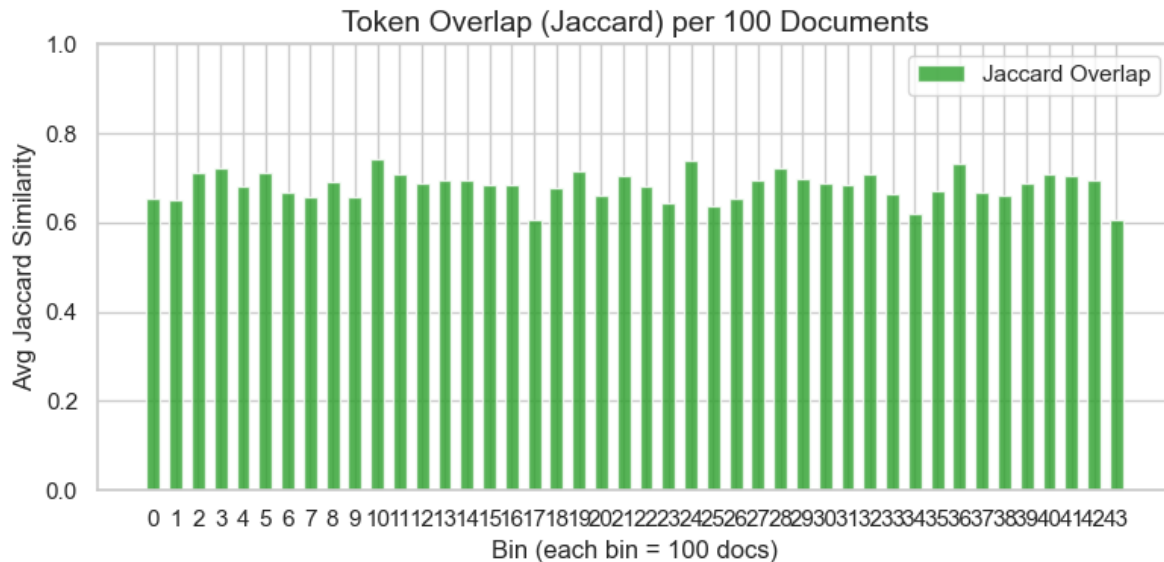
plt.title("Average Paragraph Count per 100 Documents")
plt.xlabel("Bin (each bin = 100 docs)")
plt.ylabel("Paragraph Count (mean)")
plt.xticks(bins_index)
plt.legend()
plt.tight_layout()
plt.show()
```



- **Token Overlap (Fig.3):** Jaccard similarity stays between **~0.60–0.75** (mean ~0.68), confirming both methods share core vocabulary but differ on ~30–40% of tokens, reflecting structural and boilerplate differences.

```
plt.figure(figsize=(8,4))
plt.bar(
    bins_index,
    bin_groups["jaccard"],
    width=0.6,
    color="#2ca02c",
    alpha=0.8,
    label="Jaccard Overlap"
)
```

```
plt.title("Token Overlap (Jaccard) per 100 Documents")
plt.xlabel("Bin (each bin = 100 docs)")
plt.ylabel("Avg Jaccard Similarity")
plt.ylim(0.0, 1.0)
plt.xticks(bins_index)
plt.legend()
plt.tight_layout()
plt.show()
```



These patterns are uniform across the corpus, highlighting BeautifulSoup's tendency to over-extract/segment and Docling's lean, structured output.

3.1.6 Side-by-Side Text Snippet

To complement our quantitative analysis, we examine how each parser renders the very beginning of one representative article (doc_ID:0054c566...). By comparing the first ~2,000 characters, we can directly observe differences in structure, completeness, and semantic cues.

```
def show_snippet(doc_id: str, length: int = 2000):
    bs_txt = bs_data[doc_id].get("raw_text")
    dc_txt = dc_data[doc_id].get("raw_text")
    print(f"--- Document: {doc_id} ---\n")
    print("BeautifulSoup:")
    print(bs_txt[:length] + "... \n")
    print("Docling:")
    print(dc_txt[:length] + "... \n")

# Pick a representative sample
if common_ids:
    show_snippet("0054c566353942bbdd9f9c85d9cc843aad3dce97")
```

```
--- Document: 0054c566353942bbdd9f9c85d9cc843aad3dce97 ---
```

BeautifulSoup:

In line with the Federal Council's easing of measures to contain the coronavirus, and in response to Face masks no longer compulsory outdoors
Wearing a face mask will no longer be compulsory outdoors on campus from this Saturday. However, the "The easing of these constraints gives us back some freedom," comments Ulrich Weidmann, head of the E
Easing of the requirement to wear face masks indoors
The requirement to wear a face mask indoors has been partly relaxed. Employees, researchers and students
Face masks must still be worn when moving around buildings. The same applies for discussions, meetings even if physical distancing and room occupancy requirements are observed. The only exception is small
"The pandemic is not over yet. Not everyone is fully vaccinated, and there are people who cannot be vaccinated"

Docling:

Face masks no longer compulsory outdoors

Wearing a face mask will no longer be compulsory outdoors on campus from this Saturday. However, the "The easing of these constraints gives us back some freedom," comments Ulrich Weidmann, head of the E

Easing of the requirement to wear face masks indoors

The requirement to wear a face mask indoors has been partly relaxed. Employees, researchers and students

Face masks must still be worn when moving around buildings. The same applies for discussions, meetings

even if physical distancing and room occupancy requirements are observed. The only exception is small

"The pandemic is not over yet. Not everyone is fully vaccinated, and there are people who cannot be vaccinated"

1. Lead Paragraph Omission

- **Docling** excludes the opening summary on the Executive Board's decision, possibly treating it as boilerplate metadata.
- **BeautifulSoup** preserves this lead, offering more complete context.

2. Structural Cues & Heading Detection

- **Docling** uses Markdown headings (## ...), clearly segmenting topics.
- **BeautifulSoup** leaves headings in plain text, requiring extra parsing for semantic boundaries.

3. Content Continuity & Quote Handling

- **Docling** captures full quotes (e.g., "comments Ulrich Weidmann") with no mid-sentence truncation.
- **BeautifulSoup** cuts off ("...requir..."), reducing continuity and potentially affecting NLP tasks.

4. Paragraph & Subtopic Grouping

- **Docling** aligns text into coherent subtopic blocks, aiding clarity.
- **BeautifulSoup** can fragment or merge paragraphs, needing additional logic to restore structure.

5. Downstream Workflow Considerations

- **RAG Pipelines** benefit from Docling's chunked format for precise lookups (e.g., "mask rules outdoors").

- **BeautifulSoup** may need heading detection or text segmentation before advanced tasks like QA or summarization.

6. Strategic Implications

- Clean, well-segmented text often outperforms raw extracts in NLP tasks.
- **BeautifulSoup** provides faithful raw HTML parsing.
- **Docling**'s structured output reduces post-processing overhead and improves retrieval accuracy.

Overall, **Docling** returns more structured, context-rich text, while **BeautifulSoup** stays closer to the raw HTML.

3.1.7 Pros & Cons Table

Below is a concise comparison of **BeautifulSoup** and **Docling**, reflecting key findings from the parsing experiments. In total, **4,390** HTML files were processed, with **BeautifulSoup** completing its run in **18.53 seconds** and **Docling** in **95.82 seconds**.

Aspect	BeautifulSoup	Docling
Parsing Speed	Faster startup and execution; took 18.53s for 4,390 HTML files	Slower initialization and runtime; took 95.82s for 4,390 HTML files
Dependencies	Lightweight (pure Python)	Heavier (Java + layout libraries)
Content Coverage	Includes more boilerplate; rarely misses lead paragraphs	May omit some lead or “boilerplate” content
Paragraph Structure	Splits on blank lines, leading to fragmented or excessive paragraphs	Groups logically related sentences, using headings and subheadings for coherent segmentation
Quote & Heading	Potential for partial quotes if HTML breaks unexpectedly; headings appear as plain text	Preserves quotations more reliably; injects ## Headings to mark new sections
Downstream Utility	Output often needs extra cleaning (merge paragraphs, detect headings) for advanced QA/summar.	Already chunked by headings/subtopics; minimal extra cleaning required for RAG or summarization
Overall Output	Raw HTML extraction stays close to original structure, but can be messy	Clean, structured text with more semantic detail, but risks omitting lead or navigational segments

3.1.8 Conclusion

Both parsers successfully extract content from HTML but serve different needs:

- **BeautifulSoup** excels at **fast**, broad HTML extraction. Its familiarity and low overhead make it ideal for quick data ingestion. However, it often includes boilerplate text and requires additional processing to remove noise and properly segment paragraphs.
- **Docling** produces **structured, context-rich text** with headings and coherent paragraph grouping. While it has a longer startup time (95.82s vs. 18.53s for BeautifulSoup) and can omit lead content it classifies as boilerplate, its output requires less manual cleanup for advanced NLP tasks such as RAG-based QA or summarization.

A combined **hybrid approach**—using BeautifulSoup for robust parsing and Docling’s semantic structure—will likely be the most effective way to ensure both data **completeness** and **high-quality structure** in our RAG system.

4 Step 1_2 Preprocessing

4.1 Introduction

This phase of our pipeline converts raw HTML files into structured JSON documents suitable for advanced Retrieval-Augmented Generation (RAG) tasks. It consists of two primary steps:

1. A **hybrid parsing** script (`step_1_hybrid.py`) that combines the strengths of BeautifulSoup and Docling.
2. An **advanced cleaning and metadata** script (`step_1_2_advanced_cleaning_and_metadata.py`) that refines paragraphs, removes repeated boilerplate, and enriches each document with linguistic annotations.

By the end of Step 1_2, we obtain high-quality, fully annotated text that can be indexed and chunked for downstream applications such as QA systems or summarization pipelines.

4.2 Hybrid Parsing with `step_1_hybrid.py`

4.2.1 Rationale

In order to ensure both breadth of HTML coverage and meaningful structural segmentation, **BeautifulSoup** and **Docling** are run in tandem:

- **BeautifulSoup** removes top-level non-article tags—such as `<script>`, `<style>`, `<nav>`, and `<aside>`—resulting in simpler HTML.
- **Docling** then performs deeper layout analysis on this simplified HTML, retaining headings, subheadings, and logical paragraph boundaries that might be missed by naive parsing approaches.

4.2.2 Workflow

1. **Reading & Stripping HTML:** We read each `.html` file from the `input_dir`, convert it to text with minimal tag removal, and clean up certain boilerplate elements (e.g., scripts, headers).
2. **Docling Extraction:** The cleaned HTML is fed into Docling, which captures structural cues (like headings or subtopic blocks).
3. **Normalization & Paragraph Splitting:** We normalize all Unicode characters, remove empty lines, and split the text into paragraphs either on blank lines or per line as a fallback.
4. **Output:** The script writes a minimal JSON record for each document containing:
 - `doc_id` (SHA-1 hash of the file's relative path),
 - `filename`,
 - `raw_text` (entire cleaned text),
 - `paragraphs` (a naive paragraph list).

```
{
  "doc_id": "774f...",
  "filename": "name...",
  "raw_text": "...normalized text...",
  "paragraphs": [ "para 1", "para 2", ... ]
}
```

4.2.3 Key Advantages

- **Balanced Coverage:** By removing only obvious non-content tags, we preserve critical text while eliminating distracting HTML structures.
- **Structured Paragraphs:** Docling's layout analysis often yields cleaner paragraph boundaries than naive splitting, minimizing the risk of incomplete or merged paragraphs.
- **Lightweight Output:** This script does not perform advanced tasks such as language detection or boilerplate disclaimers removal, keeping it minimal and fast for large HTML dumps.

4.3 Advanced Cleaning & Metadata with `step_1_2_advanced_cleaning_and_metadata.py`

4.3.1 Purpose

After generating minimal JSON from the hybrid parser, this second step refines and enriches the text further. Our goals include:

1. **Disclaimers & Footer Removal:** Eliminate lines that match known regex patterns (e.g., "Staffnet," "Subscribe to Newsletter," footers, disclaimers).

2. **Frequency-Based Boilerplate Removal:** Identify paragraphs repeated across many documents (e.g., repeated footers) and remove them if they exceed a predefined threshold.
3. **Metadata Extraction:** Add valuable fields (language codes, named entities, keywords, etc.) to each document.

4.3.2 Detailed Process

4.3.2.1 Boilerplate Stripping

The script first reads all JSON files output from `step_1_hybrid.py`. For each paragraph, we split it into lines, match each line against a list of **regex patterns** representing disclaimers or repeated boilerplate, and discard any matches. This produces a “refined” set of paragraphs per document.

Example regex patterns include:

```
BOILERPLATE_PATTERNS = [  
    r"Staffnet",  
    r"Newsletter\sabonnieren",  
    r"call_made",  
    ...]
```

4.3.2.2 Frequency Dictionary & Thresholding

We then build a **frequency dictionary** of refined paragraphs across all documents. Any paragraph occurring `--threshold` times (default 5) is deemed boilerplate and removed from the final output. This step ensures repeated disclaimers or common footers disappear from our text corpus.

4.3.2.3 Enrichment: Language, Entities, Keywords, Summaries

1. **Language Detection (Lingua):** Determines the main language of the refined text, crucial in multilingual contexts.
2. **Date Parsing:** If applicable, extracts dates from file path structures (e.g., 2023/05).
3. **Named Entity Recognition (spaCy):** Identifies names of persons, organizations, locations, etc.
4. **Keyword Extraction (YAKE):** Suggests key terms or phrases, facilitating topic clustering and quick lookups.
5. **Summaries:** Generates short bullet-style or sentence-based summaries, providing quick glimpses of the document content. **Note:** This can be also done via LLMs, but we opted for a less costly approach here since we are learning and not generating a production system.

4.3.2.4 Final JSON Output

For each document, the script writes:

- **main_content**: Text after disclaimers and repeated paragraphs are removed.
- **paragraphs_original** & **paragraphs_cleaned**: Preserving both the raw paragraphs from Step 1 and the newly refined paragraphs.
- **named_entities**, **keywords**, **summary**: Various NLP-enriched fields.
- **text_stats**: Basic statistics (char/word/paragraph count).
- **semantic_chunk_hints**: Indicating how the text might be split for downstream tasks (e.g., chunking per paragraph).

```
{
  "doc_id": "9b7f034dc9cd5fd1b...",
  "filename": "example.html",
  "domain": "ethz.ch",
  "language": "de",
  "title": "", //--> not implemented
  "date": "2023-05-01", //--> always YYYY-MM-01
  "year": 2023,
  "month": 5,
  "source": "ETH News", //---> always ETH News

  "main_content": "Margrit Leuthold has been the Executive ...",
  "paragraphs_original": [
    "## About the author",
    "Margrit Leuthold has been the Executive ...",
    "## Subscribe to Newsletter",
    "Subscribe to the Newsletter for internal news",
    "## Staffnet",
    "Info portal for employees ..."
  ],
  "paragraphs_cleaned": [
    "Margrit Leuthold has been the Executive ...",
    "... cleaned paragraph 2 if any ..."
  ],

  "named_entities": [
    {"text": "Margrit Leuthold", "label": "PERSON"},
    {"text": "ETH Zurich", "label": "ORG"}
  ],
  "keywords": ["Margrit Leuthold", "Bangalore", "Executive Director"],
  "summary": "Margrit Leuthold has been the Executive Director ...",
  "text_stats": {
    "char_count": 864,
    "word_count": 128,
    "paragraph_count": 1
  },
  "semantic_chunk_hints": [
    {"type": "paragraph_boundaries", "count": 1}
  ],
  "embedding_vector": [],
  "doc_embedding": []
}
```

4.3.3 Advantages of a Two-Step Approach

- **Modularity**: Splitting hybrid parsing from advanced cleanup facilitates easier debugging and iterative refinement of cleaning heuristics.

- **Reusability:** If disclaimers or threshold logic changes, you only need to re-run the second script—no need to re-parse the original HTML.
- **Improved Data Quality:** Frequency-based removal and disclaimers filtering ensure final text is less cluttered, boosting retrieval accuracy and language model performance in a RAG setup.

4.4 Putting It All Together

4.4.1 3.1 End-to-End Flow

1. Step 1: `step_1_hybrid.py`

- Converts raw HTML → minimal JSON with partial structure.
 - Note: Takes much longer than docling or BeautifulSoup alone ~231 seconds for 4,390 files.

2. Step 2: `step_2_advanced_cleaning.py`

- Cleans, deduplicates, and enriches the text → final JSON.
 - Note: Takes even longer ~1126 seconds for 4,390 files.

4.4.2 Script Usage

1. **Hybrid Parsing:** `bash python step_1_hybrid.py data/ data_cleaned/data_cleaned_BSD`
2. **Advanced Cleaning & Metadata:** `bash python step_2_advanced_cleaning.py data_cleaned/data_cleaned_BSD data_cleaned/BSD_advanced --threshold 5`

4.4.3 Outcome

By the end of Step 1_2, you have an organized corpus of documents: - Stripped of large boilerplate tags and disclaimers. - Free of high-frequency repeated paragraphs. - Enriched with metadata for quick indexing, querying, and RAG-based question-answering.

This two-phase solution forms a solid foundation for any subsequent NLP tasks, ensuring that every document is both comprehensive (retaining essential content) and clean (removing repetitive or irrelevant text).

5 Preliminary Next Steps Toward RAG Implementation (not done yet)

Below is a concise outline of what to do **after** you've completed Step 1 (HTML parsing) and Step 1_2 (advanced cleaning and metadata). **Note:** Validate & Finalize the Structured Dataset I will include in this notebook later, the rest will be done in second notebook.

5.1 Validate & Finalize the Structured Dataset

1. **Check Field Completeness:** Make sure each JSON has the following fields:
 - `doc_id`, `filename`, `domain` (e.g., "ethz.ch")
 - `language` (e.g., "en" or "de")
 - `title`, `date`, `year`, `month`, `source`
 - `main_content` (core text after disclaimers)
 - `paragraphs_original`, `paragraphs_cleaned`
 - `named_entities`, `keywords`, `summary`, `text_stats`
2. **Ensure Consistency:** Confirm that all documents follow a uniform JSON schema and that values like `language` or `paragraph_count` are valid.

5.2 Chunking & Preprocessing for Retrieval

1. **Decide on Chunk Sizes:** Split each document (via `paragraphs_cleaned` or `main_content`) into smaller segments (e.g., ~512 tokens each) for embedding or retrieval.
2. **Handle Umlauts if Needed:** If your BM25 or multilingual approach requires rewriting "ä" to "ae," consider applying it here—but only if your retrieval method benefits from it.

5.3 Index Construction & Embeddings

1. **BM25 Index:**
 - Build a *keyword-based* index from either the original or the cleaned paragraphs (`paragraphs_cleaned`).
 - Use a multilingual approach if you have both English and German documents.
2. **Vector Embeddings:**
 - Use mBERT, Sentence-BERT, or similar to convert each chunk into an embedding.
 - Store these embeddings in a vector database (e.g., FAISS, Pinecone, Chroma).

5.4 Implement Baseline & Advanced Retrieval Agents

1. **Baseline: BM25**
 - Tokenize queries (detect language, apply simple translations).
 - Retrieve top-k documents from the BM25 index.
2. **Semantic Retrieval**
 - Encode queries and chunks as vectors.
 - Compute similarity (cosine) to retrieve top-k from your vector database.
3. **GraphRAG Retrieval** (if required):
 - Build a local-to-global graph from your chunks, focusing on related paragraphs or hyperlinked references.
 - Retrieve relevant nodes from both English and German data after query translation.

5.5 Hybrid Retrieval

1. Merge BM25 + Dense Results:

- Combine top-k from each approach.
- De-duplicate or re-rank them based on certain heuristics (e.g., MRR or a custom scoring function).

2. Enhance with GraphRAG:

- If using GraphRAG, incorporate linking among the newly retrieved paragraphs to expand or refine the context.

5.6 Evaluation & Benchmarking

1. Create a Benchmark Q&A Dataset:

- Sample queries relevant to your news corpus.
- Label query-document pairs with “fully answers” (1.0), “partially answers” (0.5), or “not relevant” (0.0).

2. Calculate Retrieval Metrics:

- **Precision@k**, **Recall@k**, **MRR** to measure how well each retrieval strategy is performing.
- Compare results across baseline (BM25), semantic search, GraphRAG, and hybrid.

5.7 Final Report & Analysis

1. Discuss Observations:

- Which retrieval method ranks highest for queries in your benchmark?
- How did umlaut handling or cross-lingual queries affect performance?

2. Document Strengths & Limitations:

- E.g., BM25 struggles with synonyms or partial matches in German, while semantic search excels at conceptual matches.

3. Suggest Future Work:

- Potential expansions (e.g., better chunking, fine-tuning embeddings, advanced query rewriting).